

Module 03 — The Spine-Leaf Fabric

FOR DUMMIES

Goal: deploy the 4-node Clos fabric and bring up its OSPF underlay by hand, on every node. Yes, by hand, on purpose: module 04 automates exactly this, and the automation only lands if you've felt the repetition it removes.

1. Why spine-leaf at all

The classic three-tier design (access/distribution/core) was built for north-south traffic — clients talking to servers somewhere else. Modern data centers are dominated by east-west traffic — servers talking to each other — and three-tier gives that traffic wildly inconsistent path lengths and a redundancy story built on spanning-tree's "block half your links" model.

A Clos fabric fixes this with brute symmetry: **every leaf connects to every spine**. Any leaf-to-leaf path is exactly two hops. All links are routed and all links forward (ECMP), so adding bandwidth means adding a spine, not forklifting a core. This is what Arista built its business on, and it's the substrate VXLAN/EVPN (module 05) assumes.

Look at the diagram in `topologies/spine-leaf.clab.yml`, then at its `links:` block, and confirm they say the same thing. Four nodes, four links, full mesh between tiers.

2. The IP plan

Written down *before* configuring anything, like an adult:

What	Value	Why
spine1 Loopback0	10.255.0.1/32	Router-ID + stable endpoint. Spines low,
spine2 Loopback0	10.255.0.2/32	leaves at .11+ so the role is readable
leaf1 Loopback0	10.255.0.11/32	straight from the address.
leaf2 Loopback0	10.255.0.12/32	
spine1-leaf1	10.0.1.0/31 (spine=.0, leaf=.1)	/31 on p2p links (RFC 3021) —
spine1-leaf2	10.0.1.2/31 (spine=.2, leaf=.3)	standard modern practice, half the

What	Value	Why
spine2-leaf1	10.0.2.0/31 (spine=.0, leaf=.1)	address burn of /30s. Pattern:
spine2-leaf2	10.0.2.2/31 (spine=.2, leaf=.3)	10.0.<spine#>.x

Loopbacks matter more here than in enterprise networks: in module 05, BGP EVPN peers loopback-to-loopback and VXLAN tunnels terminate on loopbacks. The underlay's *entire job* is making these /32s reachable.

3. Deploy

```
cd ~/clab-bootstrap
./scripts/02-deploy-lab.sh topologies/spine-leaf.clab.yml
docker stats --no-stream      # meet your new RAM bill
```

4. Configure the underlay by hand

The pattern, per node: `ip routing`, fabric interfaces to routed mode with their /31s, Loopback0, OSPF with the router-id. Here's leaf1 completely; derive the other three from the IP plan table — deriving is the exercise:

```
enable
configure terminal
ip routing
!
interface Ethernet1
  description to_spine1_Et1
  no switchport
  ip address 10.0.1.1/31
  ip ospf network point-to-point
!
interface Ethernet2
  description to_spine2_Et1
  no switchport
  ip address 10.0.2.1/31
  ip ospf network point-to-point
!
interface Loopback0
  ip address 10.255.0.11/32
!
router ospf 100
  router-id 10.255.0.11
  network 10.0.0.0/8 area 0.0.0.0
  max-lsa 12000
!
end
write memory
```

Two details worth understanding rather than copying:

- `ip ospf network point-to-point` — the links *are* p2p, and telling OSPF so skips DR/BDR election: faster adjacency, less state, and it's the fabric convention everywhere.
- `network 10.0.0.0/8 area 0.0.0.0` — one statement catches loopbacks (10.255.x) and fabric links (10.0.x) both. In production you might scope tighter; in a lab, this is the right amount of typing.

Now spine1, spine2, leaf2. As you go, notice what this actually is: the same seven-ish stanzas with four values swapped — hostname-ish description, two interface IPs, loopback, router-id. Keep count of your typos. That count is module 04's sales pitch.

5. Verify like you mean it

From leaf1:

```
show ip ospf neighbor
```

Expect **two** neighbors (both spines) in `FULL`. Note there's no DR/BDR column noise — that's your point-to-point setting working. Then the test that actually matters:

```
ping 10.255.0.12 source 10.255.0.11
show ip route 10.255.0.12
```

Loopback-to-loopback, sourced correctly. The route table should show **two equal-cost paths** — one via each spine. That's ECMP, and it's the whole reason the fabric is shaped like this:

```
traceroute 10.255.0.12 source 10.255.0.11
```

Break something to prove the redundancy is real: `shutdown` leaf1's Ethernet1, confirm the ping still works (all traffic now via spine2), `no shutdown`, watch the second path return to the route table.

6. Checkpoint

Four nodes, OSPF `FULL` everywhere (leaves see 2 neighbors, spines see 2)

Any loopback pings any loopback, sourced from a loopback

`show ip route` on a leaf shows two ECMP paths to the other leaf

Failing one uplink doesn't break loopback reachability

`write memory` on all four — module 04 diffs against this config

What you learned

Clos architecture and why east-west traffic demanded it; a disciplined IP plan (/31 p2p, role-encoded loopbacks); p2p OSPF; ECMP verified by breaking things. And — the real lesson — you just typed nearly the same config four times. On four nodes that's tedious. The fabrics this design comes from have four *hundred*. Hold that thought.

Next: [Module 04 — Automation](#)